

```
#!/usr/bin/env python3
"""
pycrack.py - Python Hash Cracker
Hashcat alternative for Intel iGPU setups where hashcat fails with OpenCL errors.
Uses CPU multiprocessing - all cores, no GPU needed.
```

Usage:

```
python3 pycrack.py -m md5      -H <hash>      -w rockyou.txt
python3 pycrack.py -m sha256 -H <hash>      -w rockyou.txt -r
python3 pycrack.py -m ntlm    -H <hash>      -w rockyou.txt
python3 pycrack.py -m sha1    -f hashes.txt -w rockyou.txt
python3 pycrack.py --benchmark
```

Requires: Python 3.8+ | Zero external dependencies

```
"""
```

```
import hashlib
import sys
import os
import time
import argparse
import multiprocessing as mp
from itertools import islice
```

```
# — Supported hash modes —————
```

```
def _ntlm(data: bytes) -> str:
    """NTLM = MD4 of UTF-16LE encoded password."""
    import struct
    # Pure Python MD4 - no external dep needed
    def _md4(msg: bytes) -> bytes:
        def f(x, y, z): return (x & y) | (~x & z)
        def g(x, y, z): return (x & y) | (x & z) | (y & z)
        def h(x, y, z): return x ^ y ^ z
        def rol(x, n): return ((x << n) | (x >> (32 - n))) & 0xFFFFFFFF
        msg = bytearray(msg)
        orig_len = len(msg) * 8
        msg.append(0x80)
        while len(msg) % 64 != 56:
            msg.append(0)
        msg += struct.pack('<Q', orig_len)
        a0, b0, c0, d0 = 0x67452301, 0xEFCDAB89, 0x98BADCFE, 0x10325476
        for i in range(0, len(msg), 64):
            X = list(struct.unpack('<16I', msg[i:i+64]))
```

```

a, b, c, d = a0, b0, c0, d0
# Round 1
for k, s in zip([0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15],
                [3,7,11,19]*4):
    a = rol((a + f(b,c,d) + X[k]) & 0xFFFFFFFF, s)
    a, b, c, d = d, a, b, c
# Round 2
for k, s in zip([0,4,8,12,1,5,9,13,2,6,10,14,3,7,11,15],
                [3,5,9,13]*4):
    a = rol((a + g(b,c,d) + X[k] + 0x5A827999) & 0xFFFFFFFF, s)
    a, b, c, d = d, a, b, c
# Round 3
for k, s in zip([0,8,4,12,2,10,6,14,1,9,5,13,3,11,7,15],
                [3,9,11,15]*4):
    a = rol((a + h(b,c,d) + X[k] + 0x6ED9EBA1) & 0xFFFFFFFF, s)
    a, b, c, d = d, a, b, c
a0 = (a0 + a) & 0xFFFFFFFF
b0 = (b0 + b) & 0xFFFFFFFF
c0 = (c0 + c) & 0xFFFFFFFF
d0 = (d0 + d) & 0xFFFFFFFF
return struct.pack('<4I', a0, b0, c0, d0)
encoded = msg.encode('utf-16-le') if isinstance(msg, str) else data.decode('l
return _md4(encoded).hex()

HASH_MODES = {
    # name          : (hashlib_name or None, special_fn or None, expected_length)
    "md5"           : ("md5", None, 32),
    "sha1"          : ("sha1", None, 40),
    "sha224"        : ("sha224", None, 56),
    "sha256"        : ("sha256", None, 64),
    "sha384"        : ("sha384", None, 96),
    "sha512"        : ("sha512", None, 128),
    "sha3_256"      : ("sha3_256", None, 64),
    "sha3_512"      : ("sha3_512", None, 128),
    "ntlm"          : (None, _ntlm, 32),
    "md4"           : (None, _ntlm, 32), # NTLM uses MD4
}

LENGTH_MAP = {32: ["md5", "ntlm"], 40: ["sha1"], 56: ["sha224"],
               64: ["sha256", "sha3_256"], 96: ["sha384"], 128: ["sha512", "sha3_512"]

# — Simple mutation rules —————

def apply_rules(word: str):
    """
    Yield word variants: plain, capitalised, upper, l33t, append common digits.
    Equivalent to a small subset of hashcat's best64.rule.

```

```

"""
yield word                                # as-is
yield word.capitalize()                  # First letter upper
yield word.upper()                       # ALL CAPS
yield word.lower()                       # all lower
# Append common suffixes
for suffix in ("1","2","123","!", "1234","2024","2025","01","2023"):
    yield word + suffix
    yield word.capitalize() + suffix
# l33t speak
l33t = word.lower().replace('a','4').replace('e','3').replace('i','1').replac
if l33t != word.lower():
    yield l33t
    yield l33t.capitalize()

# — Worker function (runs in each subprocess) —————

def _worker(args):
    """
    Each worker receives a chunk of the wordlist as a list of strings.
    Returns the cracked password string, or None.
    """
    chunk, target_hash, mode, use_rules, encoding = args
    alg_name, special_fn, _ = HASH_MODES[mode]

    def compute(pw_bytes: bytes) -> str:
        if special_fn:
            return special_fn(pw_bytes)
        return hashlib.new(alg_name, pw_bytes).hexdigest()

    target = target_hash.lower()
    for line in chunk:
        word = line.rstrip('\n')
        candidates = apply_rules(word) if use_rules else [word]
        for candidate in candidates:
            try:
                pw_bytes = candidate.encode(encoding, errors='ignore')
                if compute(pw_bytes) == target:
                    return candidate
            except Exception:
                pass
    return None

# — Chunked file reader —————

def _read_chunks(filepath: str, chunk_size: int, encoding: str):
    """Lazily yield chunks of lines from a file."""

```

```

with open(filepath, 'r', encoding=encoding, errors='replace') as fh:
    while True:
        chunk = list(islice(fh, chunk_size))
        if not chunk:
            break
        yield chunk

# — Auto-detect hash mode —————

def auto_detect(h: str) -> str:
    h = h.strip().lower()
    candidates = LENGTH_MAP.get(len(h), [])
    if candidates:
        return candidates[0]
    return "md5"

# — Core crack function —————

def crack(target_hash: str, mode: str, wordlist: str, use_rules: bool,
          encoding: str = "latin-1", workers: int = None, chunk_size: int = 50_00)

    target_hash = target_hash.strip().lower()

    if mode not in HASH_MODES:
        print(f"[!] Unknown mode '{mode}'. Available: {'', '.join(HASH_MODES)}")
        return None

    if not os.path.exists(wordlist):
        print(f"[!] Wordlist not found: {wordlist}")
        return None

    n_workers = workers or max(1, mp.cpu_count() - 1)
    total_size = os.path.getsize(wordlist)

    print(f" Mode      : {mode.upper()}")
    print(f" Target    : {target_hash}")
    print(f" Wordlist   : {wordlist} ({total_size/1_048_576:.1f} MB)")
    print(f" Workers    : {n_workers} CPU cores")
    print(f" Rules      : {'ON (mutations enabled)' if use_rules else 'OFF'}")
    print(f" Starting crack... (Ctrl+C to stop)\n")

    start      = time.time()
    tried      = 0
    found      = None

    with mp.Pool(n_workers) as pool:
        try:

```

```

        for result in pool.imap_unordered(
            _worker,
            ((chunk, target_hash, mode, use_rules, encoding)
             for chunk in _read_chunks(wordlist, chunk_size, encoding)),
            chunksize=1
        ):
            tried += chunk_size
            elapsed = time.time() - start
            rate = tried / elapsed if elapsed > 0 else 0
            print(f" {tried:>12,} words | {rate:>10,.0f} w/s | {elapsed:}
                  if result is not None:
                        found = result
                        pool.terminate()
                        break
    except KeyboardInterrupt:
        pool.terminate()
        print("\n\n [!] Stopped by user")
        return None

elapsed = time.time() - start
print()
if found:
    print(f"\n ✓ CRACKED! Password = \033[92m\033[1m{found}\033[0m")
    print(f"      Hash      = {target_hash}")
    print(f"      Time       = {elapsed:.2f}s | ~{tried:,} words tried")
else:
    print(f"\n [!] Not found after {tried:,} words ({elapsed:.2f}s)")
    print(f"      Try: -r (rules) | different wordlist | john --rules")
return found

# — Multi-hash file mode —————

def crack_file(hashfile: str, mode: str, wordlist: str, use_rules: bool, encoding
with open(hashfile) as hf:
    hashes = [h.strip() for h in hf if h.strip()]
    print(f" Loaded {len(hashes)} hashes from {hashfile}")
    results = {}
    for i, h in enumerate(hashes, 1):
        print(f"\n [{i}/{len(hashes)}] Cracking: {h}")
        result = crack(h, mode, wordlist, use_rules, encoding)
        results[h] = result
    print(f"\n\n — SUMMARY —")
    cracked = {h: p for h, p in results.items() if p}
    print(f" Cracked: {len(cracked)}/{len(hashes)}")
    for h, p in cracked.items():
        print(f" {h[:20]}... → \033[92m{p}\033[0m")
    if cracked:

```

```

        out = "cracked.txt"
        with open(out, "w") as f:
            for h, p in cracked.items():
                f.write(f"{h}:{p}\n")
        print(f"\n Results saved to {out}")

# — Benchmark —————

def benchmark():
    print(" Benchmarking hash speed on this CPU...\n")
    test_pw = b"benchmark_password_123"
    n_workers = max(1, mp.cpu_count() - 1)
    for mode, (alg, special, _) in HASH_MODES.items():
        if special:
            fn = special
            fn2 = lambda pw, s=special: s(pw)
        else:
            fn2 = lambda pw, a=alg: hashlib.new(a, pw).hexdigest()
        start = time.time()
        count = 0
        while time.time() - start < 1.0:
            fn2(test_pw); count += 1
        rate = count * n_workers # extrapolate to all cores
        print(f" {mode:<12} ~{rate:>12,} H/s (1 core × {n_workers} workers ≈ {
print(f"\n CPU cores available: {mp.cpu_count()}")
print(f" Workers used : {n_workers}")
print(f"\n Note: hashcat on GPU is 100-1000x faster for MD5/NTLM.")
print(f" For Intel iGPU: try hashcat -D 1 (force CPU OpenCL)")
print(f" or install: sudo apt install intel-ocl-icd ocl-icd-ocl-dev")

# — CLI —————

def main():
    p = argparse.ArgumentParser(
        description="pycrack - Python hash cracker (hashcat alternative for Intel
        formatter_class=argparse.RawDescriptionHelpFormatter,
        epilog=""

Examples:
python3 pycrack.py -m md5 -H 5f4dcc3b5aa765d61d8327deb882cf99 -w rockyou.txt
python3 pycrack.py -m sha256 -H <hash> -w rockyou.txt -r
python3 pycrack.py -m ntlm -H <hash> -w rockyou.txt
python3 pycrack.py -m sha1 -f hashes.txt -w rockyou.txt
python3 pycrack.py --benchmark
python3 pycrack.py --list-modes

Supported modes:
md5, sha1, sha224, sha256, sha384, sha512, sha3_256, sha3_512, ntlm

```



```
mode    = auto_detect(sample)
print(f"  Auto-detected mode: {mode}")

if args.hashfile:
    crack_file(args.hashfile, mode, args.wordlist, args.rules, args.encoding)
else:
    crack(args.hash, mode, args.wordlist, args.rules, args.encoding, args.thr

if __name__ == "__main__":
    main()
```